# Labelling philosophical texts: methods for the project lead

This condenses the reading list to what changes how the project runs. The labeller's side (scheme, heuristics, tool how-to) is in `guide.typ`; this file covers what the labeller doesn't need: scheme design, evaluation, automation, the player side, and how to set up and run INCEpTION.

## 1. The loop

Annotation is a cycle, not a one-off pass: **model → annotate → train → test → evaluate → revise**. Pustejovsky & Stubbs call it MATTER. Your first scheme will be wrong, and finding out where is what the pilot is for. Plan to revise the labels at least twice before automating anything.

## 2. Decide before labelling

- **Keys later.** The keys don't exist yet, so the pilot tests the workflow, not the final scheme: label for a careful reader, using the heuristics in the labelling guide (§4). Once convos exist, check which labels the keys actually use; the rest are optional.
- **Layers.** A few, each answering one question: concept, argument, move, reference (labelling guide §3). Reference is where cross-document links live: Carnap quoting Heidegger.
- **Unit.** For each layer, decide what a span is: a sentence, a clause, or "as long as the idea runs". Unclear boundaries are the main source of disagreement on spans.
- **Small label sets.** Fewer than 10 labels per layer to start. Split a label later, once you can see it doing two jobs.

## 3. Write guidelines while you annotate

For each label, the labelling guide should give:

- a one-line definition;
- what's **in**, what's **out**, and the nearest label it gets confused with;
- 2–3 real examples plus one hard case.

Version the guidelines and keep a changelog. Record which version each label was made under (the `Pass` layer, §10), so you know what to re-check when a rule changes.

Philosophy-specific rules the guide must settle before the pilot:

- **Mention vs. discussion.** Settled by the `use` feature: every occurrence can be tagged, and `use` says how much the passage is about it.
- **Author-relative senses.** Concepts are a tree in a local knowledge base: senses sit under a broad concept, and spans link to the narrowest one. Keys can then match either level. See the labelling guide §3.
- **Explicit vs. reconstructed arguments.** Carnap states his premises. With Heidegger the labeller will be inferring them; the `reconstructed` feature marks that.

## 4. Disagreement is data

With one labeller, their "second annotator" is themselves, a week later (the `-reread` account, §10). With a new labeller, the lead is a second annotator too, on the calibration and overlap documents.

- Re-label a sample of passages blind and compare.
- Where the two passes disagree, sort the case into one of two kinds:
  - **The guideline is unclear.** Fix the guideline. This is the Oortwijn/Betti approach.
  - **The text really supports both readings.** Allow both labels, or add an `ambiguous` marker. This is the Gius & Jacke approach.
- Expect lower agreement on Heidegger's argument layer than on Carnap's. That's a finding about the texts and a reason the **move** layer exists, not a failure.

- Measure boundaries and labels separately: "same label, slightly different span" is a much smaller problem than "different label".

## 5. Keep the evaluation honest

- **Three splits:** explore (play freely), select (choose between prompts, thresholds, models), evaluate (look at it once, at the end). Tuning on what you report makes the numbers look better than they are.
- **Split by passage or document, not by note.** Notes on the same passage leak into each other.
- Pick a fixed gold set before you start automating, and don't edit it to match the model.

## 6. Automating: manual, then suggestions, then review

1. **Bootstrap cheaply.** Use keyword and phrase rules ("das Nichts" → **nothing**) and few-shot embeddings from your first labels. They're crude, which is fine: they only need to make suggestions.
2. **Suggest, don't apply.** The labeller accepts or rejects each one. Acceptance rate per label is your first quality measure.
3. **Guard against anchoring.** People accept wrong suggestions, especially when correcting one costs effort. Keep a blind sample labelled **without** suggestions, and compare the model against that sample, not against what was accepted.
4. **Validate each task separately.** A labeller that's good at concepts tells you nothing about arguments. Check each layer against gold.
5. **Treat the model as an instrument.** Record model, prompt or config, version and date with every automated label, and fix the settings. Watch for **systematic** error, like consistent over-tagging, more than for random error. For you, over-tagging means keys unlock too easily.
6. **Look at what to label next.** Passages where the model is least sure, or where suggestions disagree, are worth more of the labeller's time than ones it already gets right.

## 7. The player side

The player side (notes and free replies) needs its own gold set, separate from the document labels:

- **Real notes.** Write them as a player would: terse, wrong, sarcastic, half-thoughts. Gold is the key(s) each note should unlock, possibly none.
- **Fuzzing.** Include off-topic, hostile, nonsense and gaming-the-system input. These are the Façade/"melon" cases: the gold answer is "no key", and the cost of getting it wrong falls on the NPC.
- For design 2 (free replies), the gold also depends on **where in the convo** the reply happens, so record the yarn node with every reply.

## 8. When the convo changes

Keys come first, so convo edits ripple downstream:

- Treat `keys.toml` as versioned.
- When a key is added, split or removed, re-check the labels and gold notes that used it, and nothing else.
- Per-label guideline versions (§3) make that re-check targeted instead of total.

## 9. How much to label, and by whom

Manual labelling comes first; there's no way around it. But there are three corpora here, and they need very different amounts of work:

1. **The game's documents.** These get labelled regardless, because they ship. If the game has 20–50 texts, labelling all of them by hand may be cheaper than building and validating an automated

labeller. Automation pays off only if there are many documents or they change often. To find out, time the pilot (pages per hour, per layer) and extrapolate.

2. **Training data for an automated labeller.** Needed only if (1) says to automate. The game's documents double as this corpus, so you don't need a separate general one. Rough orders of magnitude:
   - few-shot (embeddings, GLiNER-style): tens of examples per label;
   - fine-tuning: hundreds;
   - an evaluation set: a few dozen per label before differences between arms mean anything.
3. **Player notes and replies.** This is the corpus that needs volume. It has to cover how people actually write, Façade cases included, and it comes from your own fakes, fuzzing and playtests, not from books.

**Analytic and continental texts together** stress-test the scheme: a layer that only works for one tradition shows up immediately.

**One reading or several:**

- **One reading (yours)** gives consistent labels. For a game that's arguably right: NPCs respond to the author's interpretation, so your reading is the ground truth by design. With a separate labeller, curation (§10) is where your reading is enforced.
- **Specialists** may read their tradition better. But if each one labels only their own tradition, you can't tell "Heidegger is harder" from "this labeller labels differently". If you bring them in, everyone labels a shared set of overlap passages from both traditions.
- Decide after the pilot. If self-agreement is fine on Carnap and poor on Heidegger, that tells you whether you need a specialist or a new layer.

## 10. INCEpTION setup

What the labelling guide assumes is already in place. Section references are to the INCEpTION 42.1 user guide.

**Instance and accounts**

- Run the standalone JAR (Java 21+) and open `http://localhost:8080`. For a remote labeller, set it up on a server instead (INCEpTION Admin Guide).
- Accounts, by project role:
  - **you**: manager + curator + annotator. A manager isn't automatically an annotator, and instance `ROLE_ADMIN` doesn't imply `ROLE_USER`.
  - **labeller**: annotator.
  - **labeller-reread**: annotator, used only for the blind re-annotation sample. INCEpTION keeps each user's annotations separate, so the second pass can't see the first.
- Workload: static assignment, so documents can be locked per user. Lock every non-sample document for `-reread`. Leave *Reopenable by annotators* on during the pilot.

**Documents**

- Import the exact UTF-8 text the player reads, as **Plain Text** (`text`). INCEpTION does its own sentence and token detection (Java `BreakIterator`), so check in the pilot how it tokenises emphasis markers if the text keeps them.
- Never commit or share project archives: they contain `source/`, i.e. the book texts. Commit the layer exports (JSON, per layer) and tagset exports instead; they contain no text.

**Decision needed:** one document per text, or one per section? Workload, the re-annotation sample, agreement and curation all work per document, so sections make a 10% sample possible

and keep pages small; the converter adds each section's offset back. The cost is that relations can't cross documents, so an argument spanning sections loses its arc. Recommendation: sections.

**Layers**

All are custom span layers. Behaviours: overlap allowed (stacking too), *Validation: Always*. Every span layer gets three shared features: `unsure` (boolean), `note` (multi-row string: the doubt, the label you nearly chose, or a reconstructed premise's wording) and `continues` (link feature to the same layer, role labels off) for interrupted passages, since INCEpTION has no discontinuous spans (FAQ).

| layer | granularity | own features |
|---|---|---|
| Concept | token, crosses sentences | `concept` (KB concept feature, any entry in the project's knowledge base) · `use` (tagset `defined`/`discussed`/`mentioned`) |
| Argument | token, crosses sentences | `label` (tagset from guide §3) · `voice` (`author`/`reported`) · `reconstructed` (boolean) |
| Move | sentence, crosses sentences | `label` (tagset from guide §3) |
| Reference | token | `label` (`quote`/`citation`/`allusion`/`other`) · `target` (string; tagset of work ids, annotators may add) · `stance` (`agrees`/`disagrees`/`neutral`) |
| Pass | document metadata | `layer` · `guideline_version` · `date` · `minutes` |

- **Argument relation**: a relation layer attached to Argument, crossing sentences. Features: `label` (`inference`/`conflict`/`rephrase`, IAT's names), `strength` (`deductive`/`defeasible`), `kind` (`rebut`/`undercut`/`dismiss`), `group` (free string for linked premises). IAT's undercut targets an inference, not a span; INCEpTION can't do that, so `kind` approximates it. A span layer can only have one relation layer, and relations can't connect layers or documents.
- **Pass** replaces a separate session log: one entry per document per layer pass, so a guideline change maps to the passes made under the old version. Document-level annotations only export in UIMA CAS JSON or XMI, which is the format used here anyway.
- Hidden premises are zero-width Argument spans (Shift+click) with `reconstructed` set and the premise written in `note`.
- No recommenders in the pilot. Later: the String Matcher only works on layers that don't cross sentences, so automating concept mentions needs either a separate token-level mention layer or the search sidebar's bulk create. Run the blind sample in a project without recommenders.
- Upload `guide.pdf` under Guidelines (PDF only); labellers open it from the book icon.

**Agreement**

- **Agreement** page, per layer and feature. For spans use **Gamma (γ)**, which aligns spans so boundary and label disagreement both count. Avoid Cohen's and Fleiss' kappa: they silently drop positions only one side annotated, which inflates the score. Read the *Notes on the result* panel before trusting a number.
- Limits: stacked annotations and zero-width spans (hidden premises) are skipped; relations only get coding measures. For labels separately from boundaries (§4), export via *Export… → CSV/Diagnosis* and compute γcat offline (`pygamma-agreement`, the library INCEpTION's γ follows).

**Curation and export**

- Curate every finished document, in integrated curation mode. This is where your reading is applied and guideline gaps show up. On sample documents both passes appear, so curating them is the disagreement resolution of §4.
- Export **curated documents** as **UIMA CAS JSON 0.4.0**. It keeps every layer, feature, relation and document-level annotation, and Python reads it with `dkpro-cassis`. The converter to our own format runs offline, so the game (Rust) never reads INCEpTION's format. Check whether its offsets are code points or UTF-16 units before converting: INCEpTION documents UTF-16 offsets for its TSV format but doesn't say for JSON.

```
INCEpTION → CAS JSON → converter (Python, offline) → our format → game
```

## 11. Running the pilot

The labeller's half of this is in the labelling guide §2.

1. **Calibrate.** You and the labeller label the same one or two sections independently (one Carnap, one Heidegger), then curate them together. Fix the guideline from what comes up, before the pilot proper.
2. **Pick the sample.** Choose the re-annotation documents (about 10%) at random, in advance, and lock everything else for `-reread`.
3. **Curate as documents finish**, within a few days, while the labeller still remembers their doubts. Bump the guideline version for every rule change and tell the labeller what changed.
4. **Re-annotate.** At least a week after a sample document is finished, the labeller does it again as `-reread`.
5. **Review.** Per layer: γ between the passes, `unsure` and doubts per page, the `other` rate, pages per hour (from `Pass`). Then revise the scheme (§1).

## Checklist before the pilot

- [ ] Unit/boundary rule per layer, written into the labelling guide
- [ ] Guide decisions settled (concept naming and tree depth, text handling)
- [ ] Guideline versioned, with a changelog; PDF uploaded
- [ ] Layers, tagsets and accounts set up (§10); layer JSON committed
- [ ] Document granularity decided; sample chosen; `-reread` locked out of the rest
- [ ] Splits by passage, eval split untouched
- [ ] Calibration sections done and curated

## Sources

By section. Links are to free copies (ACL Anthology, arXiv, publishers' open pages) and were checked on 2026-10-20. Entries marked † have no free copy linked: they are books or paywalled, and were cited from memory, so confirm title and venue before citing them. Anything not listed is a rule of thumb from these notes, not from the literature: the margin-note, summary and player tests, the density check, the 10–15% `other` threshold, the shortest-span rule, the session length, the warm-up, the `restates` observation about continental texts, and the corpus sizes in §9.

**§1 The loop**

- Pustejovsky, Stubbs. *Natural Language Annotation for Machine Learning*. O'Reilly, 2012. † MATTER cycle.
- Ide, Pustejovsky (eds.). *Handbook of Linguistic Annotation*. Springer, 2017. † Reference for everything here.

**§2 Layers, units, label sets**

- Hovy, Lavid. “Towards a ‘science’ of corpus annotation: a new methodological challenge for corpus linguistics.” *Int. J. of Translation*, 2010. † The core decisions: unit, labels, guidelines, measurement.
- Fort. *Collaborative Annotation for Reliable Natural Language Processing*. Wiley/ISTE, 2016. † Scheme design and project practice.

**§3 Guidelines; philosophy-specific rules**

- Reiter, Willand, Gius. “A Shared Task for the Digital Humanities”, ch. 1–3. *J. of Cultural Analytics* 4(3), 2019. How to write and evaluate guidelines for an interpretive phenomenon. https://doaj.org/article/8839c26c223245a1a2d7abcd898475c2
- Betti, van den Berg. “Towards a Computational History of Ideas.” CEUR-WS vol. 1681, 2016. Concepts as explicit expert models; author-relative senses. https://ceur-ws.org/Vol-1681/Betti_van_den_Berg_computational_history_of_ideas.pdf
- Betti, Reynaert, Ossenkoppele, Oortwijn, Salway, Bloem. “Expert Concept-Modeling Ground Truth Construction for Word Embeddings Evaluation in Concept-Focused Domains.” COLING 2020. Gold standard for one philosopher’s concepts. https://aclanthology.org/2020.coling-main.586/
- Becker, Paul, Frank. “Implicit Knowledge in Argumentative Texts: An Annotated Corpus.” LREC 2020. Annotating unstated premises. https://aclanthology.org/2020.lrec-1.282/
- Betz, Richardson. “DeepA2: A Modular Framework for Deep Argument Analysis with Pretrained Neural Text2Text Language Models.” *SEM 2022. Reconstruction linked back to the source text. https://aclanthology.org/2022.starsem-1.2/

**§4 Disagreement**

- Gius, Jacke. “The Hermeneutic Profit of Annotation: On Preventing and Fostering Disagreement in Literary Analysis.” *IJHAC* 11(2), 2017. https://doi.org/10.3366/ijhac.2017.0194
- Oortwijn, Ossenkoppele, Betti. “Interrater Disagreement Resolution: A Systematic Procedure to Reach Consensus in Annotation Tasks.” HumEval @ EACL 2021. https://aclanthology.org/2021.humeval-1.15/
- Plank. “The ‘Problem’ of Human Label Variation: On Ground Truth in Data, Modeling and Evaluation.” EMNLP 2022. https://aclanthology.org/2022.emnlp-main.731/
- Artstein, Poesio. “Inter-Coder Agreement for Computational Linguistics.” *Computational Linguistics* 34(4), 2008. https://aclanthology.org/J08-4004/
- Mathet, Widlöcher, Métivier. “The Unified and Holistic Method Gamma (γ) for Inter-Annotator Agreement Measure and Alignment.” *Computational Linguistics* 41(3), 2015. Agreement on boundaries and labels together. https://aclanthology.org/J15-3003/
- Mathet. “The Agreement Measure γcat, a Complement to γ Focused on Categorization of a Continuum.” *Computational Linguistics* 43(3), 2017. Labels separately from boundaries. https://aclanthology.org/J17-3006/

**§5 Evaluation**

- Fang, Garcia Bernardo, van Kesteren. “A Methodological Guide on Using Large Language Models for Text Annotation in the Social Sciences and Humanities with Python and R.” arXiv 2604.09638, 2026. Explore/select/evaluate splits. https://arxiv.org/abs/2604.09638
- Cawley, Talbot. “On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation.” *JMLR* 11, 2010. Why tuning on what you report inflates it. https://jmlr.org/papers/v11/cawley10a.html
- Kaufman, Rosset, Perlich, Stitelman. “Leakage in Data Mining: Formulation, Detection, and Avoidance.” *ACM TKDD* 6(4), 2012. Why to split by passage. https://doi.org/10.1145/2382577.2382579

**§6 Automating**

- Klie, Eckart de Castilho, Gurevych. “From Zero to Hero: Human-In-The-Loop Entity Linking in Low Resource Domains.” ACL 2020. Suggestions from scratch. https://aclanthology.org/2020.acl-main.624/
- Ratner et al. “Snorkel: Rapid Training Data Creation with Weak Supervision.” VLDB 2017. Rule-based bootstrapping. https://arxiv.org/abs/1711.10160
- Tunstall et al. “Efficient Few-Shot Learning Without Prompts” (SetFit). arXiv, 2022. Few-shot embeddings. https://arxiv.org/abs/2209.11055
- Zaratiana, Pasternak, Boyd, Hurn-Maloney, Lewis. “GLiNER2: An Efficient Multi-Task Information Extraction System with Schema-Driven Interface.” arXiv 2507.18546, 2025. https://arxiv.org/abs/2507.18546
- Berzak, Huang, Barbu, Korhonen, Katz. “Anchoring and Agreement in Syntactic Annotations.” EMNLP 2016. Anchoring on pre-annotations. https://aclanthology.org/D16-1239/
- Fort, Sagot. “Influence of Pre-annotation on POS-tagged Corpus Development.” Linguistic Annotation Workshop, 2010. https://aclanthology.org/W10-1807/
- Beck et al. “Bias in the Loop: How Humans Evaluate AI-Generated Suggestions.” arXiv 2509.08514, 2025. https://arxiv.org/abs/2509.08514
- Pangakis, Wolken, Fasching. “Automated Annotation with Generative AI Requires Validation.” arXiv 2306.00176, 2023. Validate each task. https://arxiv.org/abs/2306.00176
- Egami, Hinck, Stewart, Wei. “Using Imperfect Surrogates for Downstream Inference: Design-based Supervised Learning for Social Science Applications of Large Language Models.” NeurIPS 2023. Systematic error in machine labels. https://arxiv.org/abs/2306.04746
- Settles. “Active Learning Literature Survey.” UW–Madison CS tech report 1648, 2009. What to label next. https://burrsettles.com/pub/settles.activelearning.pdf
- Klie, Webber, Gurevych. “Annotation Error Detection: Analyzing the Past and Present for a More Coherent Future.” *Computational Linguistics* 49(1), 2023. Finding your own mistakes. https://aclanthology.org/2023.cl-1.4/

**§7 The player side**

- Mateas, Stern. “Façade: An Experiment in Building a Fully-Realized Interactive Drama.” Game Developers Conference, 2003. †
- Mateas, Stern. “Natural Language Understanding in Façade: Surface-text Processing.” TIDSE 2004. † How Façade mapped free text to discourse acts; the closest prior art for NPC design 2.

**§10 INCEpTION**

- INCEpTION User Guide, release 42.1. https://inception-project.github.io/releases/42.1/docs/user-guide.html
- Apache UIMA, JSON CAS specification, https://github.com/apache/uima-uimaj-io-jsoncas/blob/main/SPECIFICATION.adoc. `dkpro-cassis`, https://pypi.org/project/dkpro-cassis/.
- W3C. *Web Annotation Data Model.* W3C Recommendation, 2017, https://www.w3.org/TR/annotation-model/.
- Markup and offsets: TEI Consortium. *TEI P5 Guidelines*, ch. 20 “Non-hierarchical Structures” (overlapping spans vs. tree markup), https://tei-c.org/release/doc/tei-p5-doc/en/html/NH.html. Rust parsers reporting source ranges: `pulldown-cmark` (offset iterator), https://docs.rs/pulldown-cmark/latest/pulldown_cmark/struct.OffsetIter.html, and `typst-syntax`, https://docs.rs/typst-syntax/latest/typst_syntax/.


Sources for the scheme and the labelling heuristics are listed in the labelling guide.